

미래형자동차 핵심기술 전문인력양성사업

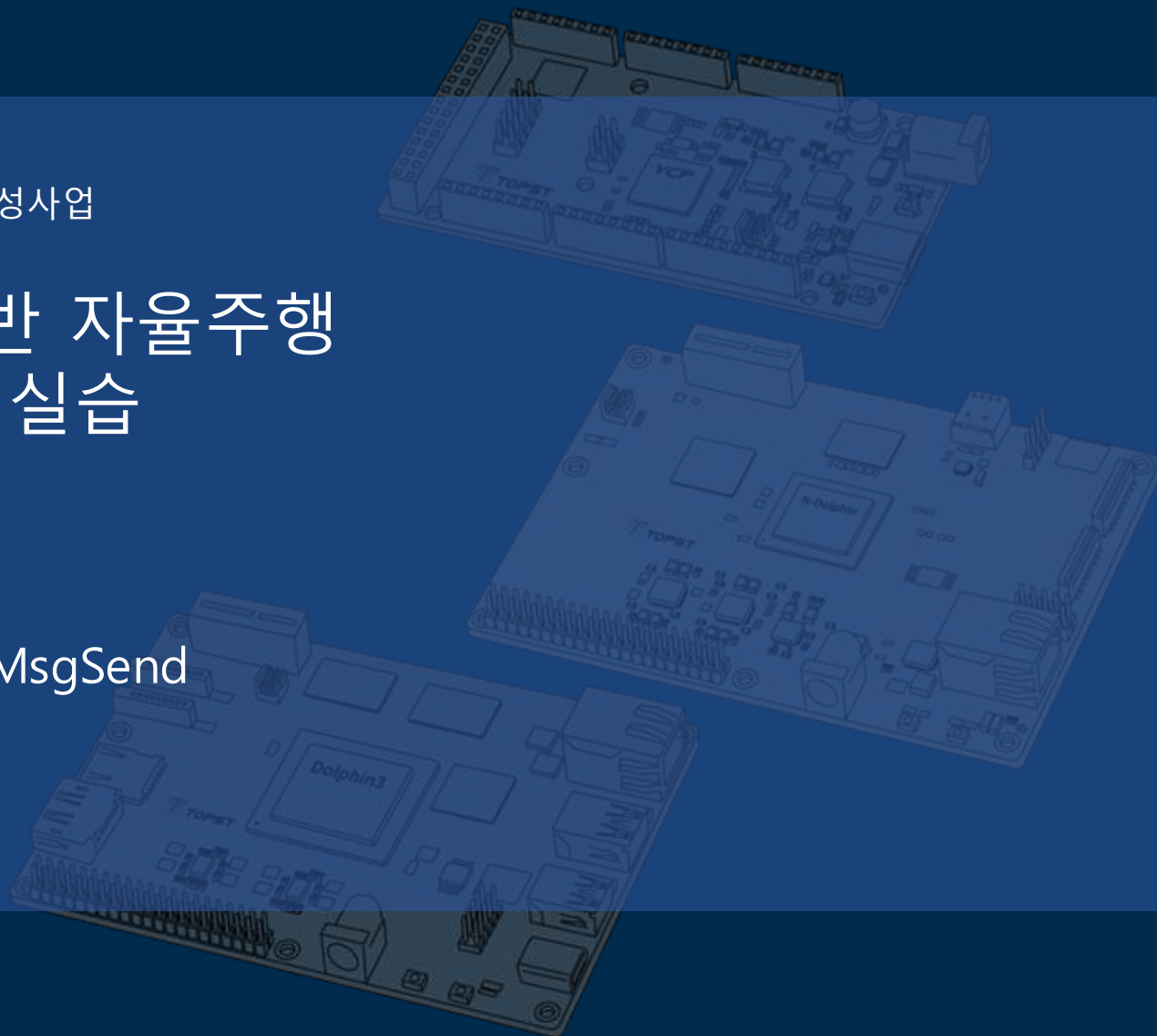
SDV 아키텍처 기반 자율주행 통합 시스템 구현 실습

● ● ●

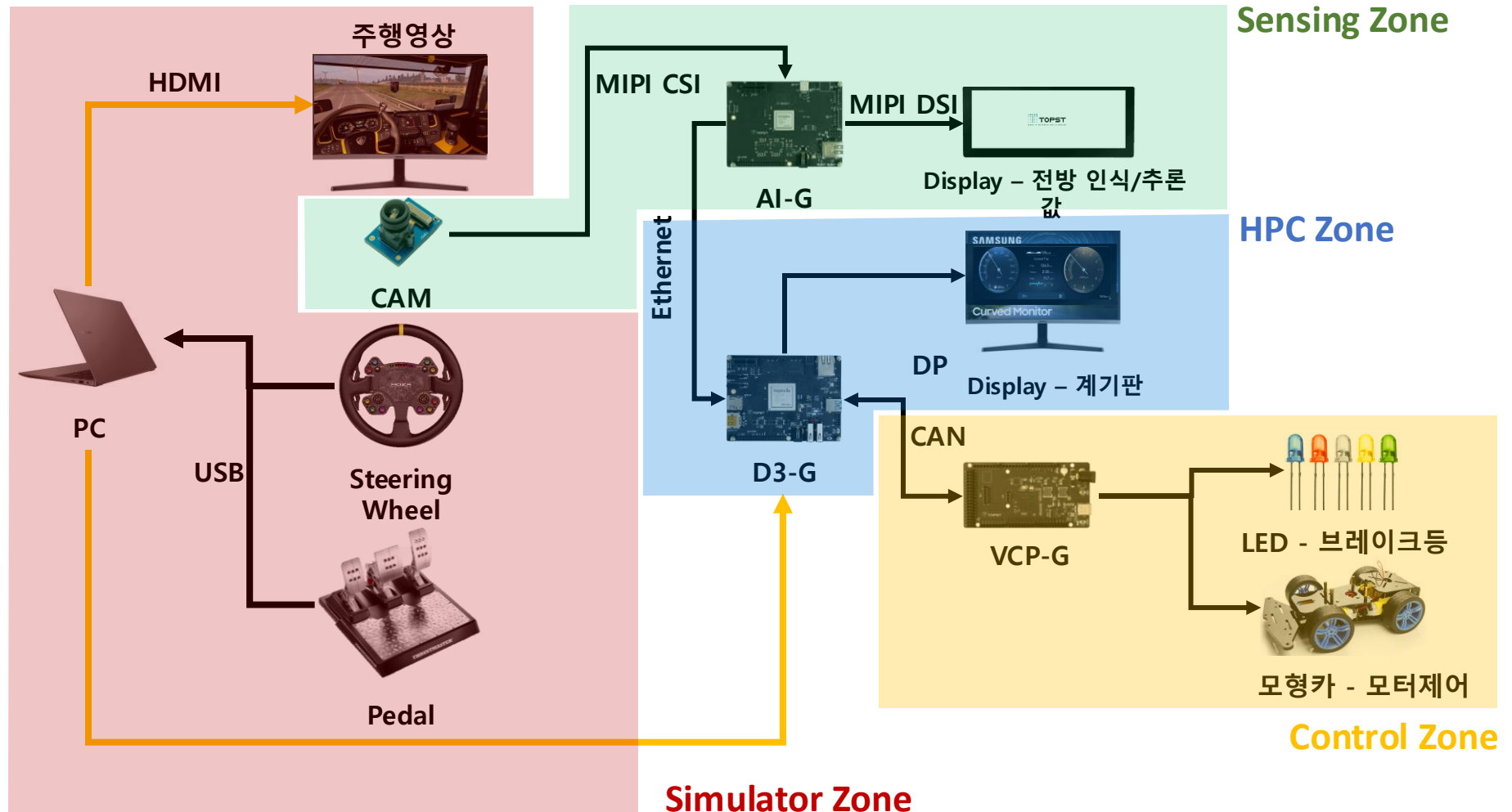
3일차

4 HPCZone & CANMsgSend

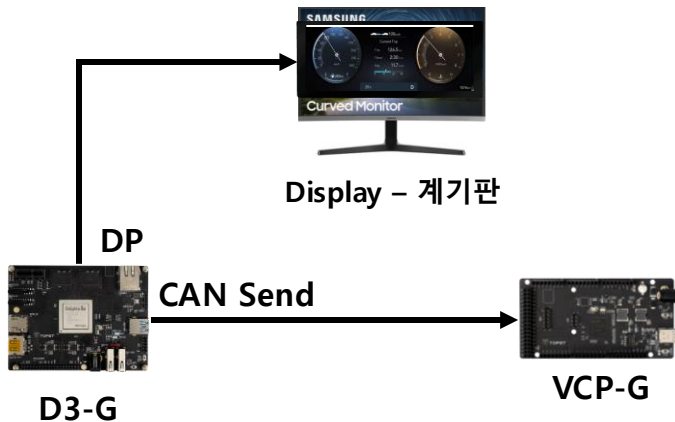
Telechips TOPST



Zonal 기반 차량 인터랙션 시스템-전체 시나리오



HPC Zone 구현



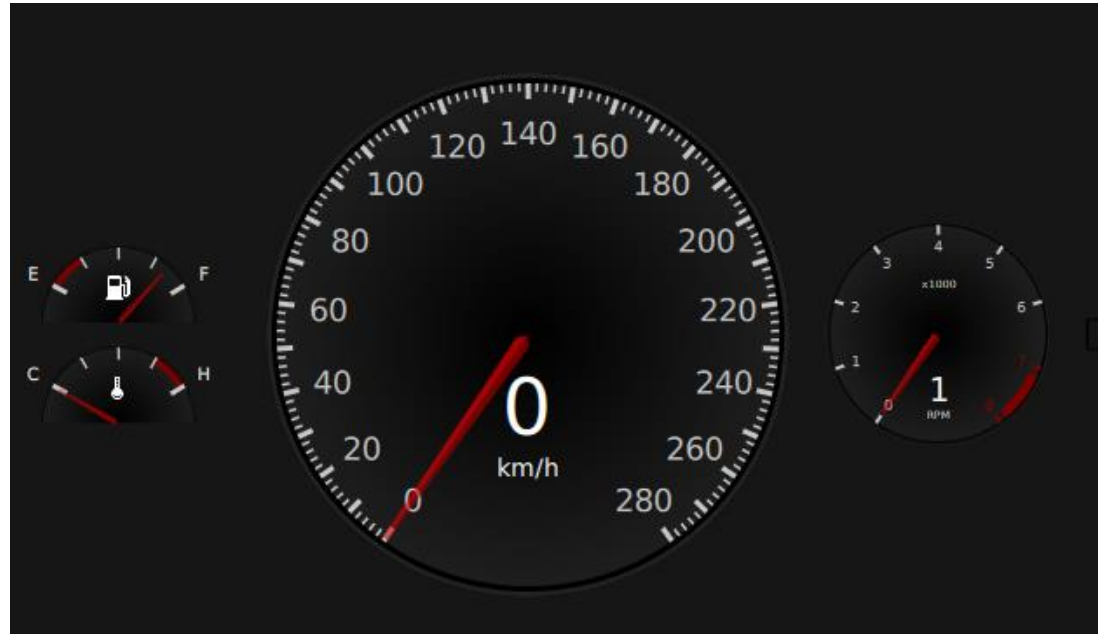
Zone Description

- 계기판 애플리케이션을 통해 시뮬레이터 차량 상태 확인
- D3-G 내 정의된 CAN 메시지 구성 및 송신

Task Goal

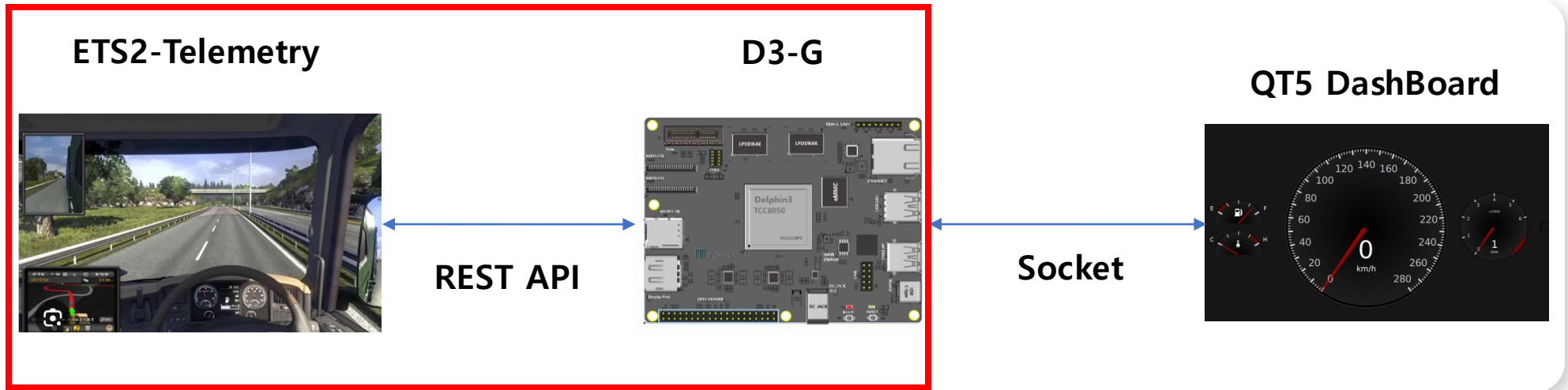
- Dashboard UI
- Dashboard Data Process
- CAN
- Handling CAN message on D3-G

Dashboard UI



- Based on Qt5
- 계기반에 표시할 항목 선정
ex) speed, rpm, fuel

Dashboard Data Process



- Telemetry를 통해 차량의 상태 값을 실시간으로 받아 옴
- D3-G 에서 HTTP 프로토콜로 서버에 연결
- GET 메서드를 통해 값을 호출

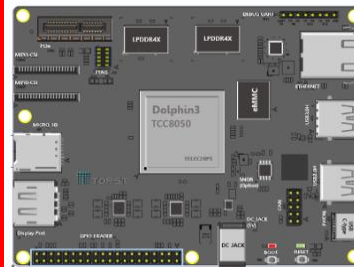
Dashboard Data Process

ETS2-Telemetry



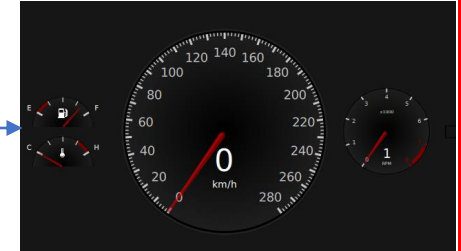
REST API

D3-G



Socket

QT5 DashBoard



```
def serve():
    global _stop
    srv = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind((HOST, PORT))
    srv.listen(1)
    print(f"[NET] listening on {HOST}:{PORT}")

    while not _stop:
        conn, addr = srv.accept()
        print(f"[NET] client connected: {addr[0]}:{addr[1]}")
        try:
            conn.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)
            period = 1.0 / SEND_HZ
            next_t = time.monotonic()
            encoder = json.dumps
            while True:
                with _lock:
                    payload = dict(_latest)
                    line = encoder(payload, separators=(",", ":")) + "\n"
                    conn.sendall(line.encode("utf-8"))

                next_t += period
                now = time.monotonic()
                if next_t > now:
                    time.sleep(next_t - now)
                else:
                    next_t = now
```

python script

- D3-G에서 QT Dashboard와 소켓 통신
- 원하는 값만 추출하여 json 파일 형태로 QT에 전송
- QT에서는 signals를 통해 받아온 값을 qml로 전송
- qml에서 받아온 값들을 활용해 rpm, speed, fuel 값들을 실시간으로 시각화

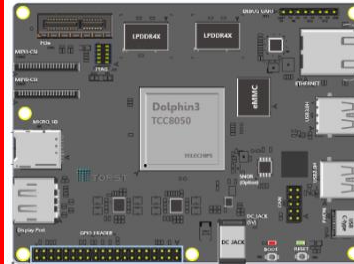
Dashboard Data Process

ETS2-Telemetry



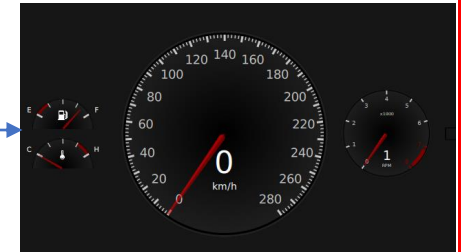
REST API

D3-G



Socket

QT5 DashBoard



```
#include <QDebug>
#include <QtGui/QtFont>
#include <QtGui/QtFontDatabase>
#include <QJsonDocument>
#include <QJsonObject>

class TcpClient : public QObject {
    Q_OBJECT
public:
    explicit TcpClient(QObject* parent=nullptr) : QObject(parent) {
        connect(&socket, &QTcpSocket::connected, this, []{
            qDebug() << u8"서버에 연결 성공!";
        });
        connect(&socket, &QTcpSocket::disconnected, this, []{
            qDebug() << u8"서버와 연결이 끊어졌습니다.";
        });
        connect(&socket, &QTcpSocket::readyRead, this, &TcpClient::onReadyRead);
        socket.connectToHost("192.168.137.31", 9999);
    }

signals:
    void telemetryUpdated(double rpm, double speedKmh, double fuelL, double steering);
};
```

QT cpp code – TcpClient Class

- D3-G에서 QT Dashboard와 소켓 통신
- 원하는 값만 추출하여 json 파일 형태로 QT에 전송
- QT에서는 signals를 통해 받아온 값을 qml로 전송
- qml에서 받아온 값들을 활용해 rpm, speed, fuel 값들을 실시간으로 시각화

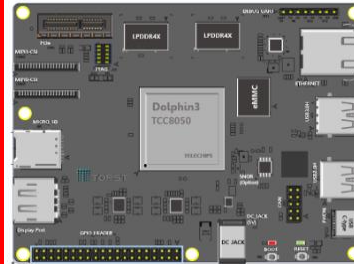
계기판 DP 구성 - QT

ETS2-Telemetry



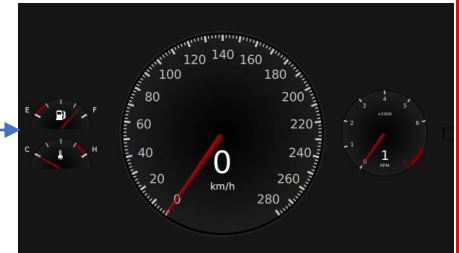
REST API

D3-G



Socket

QT5 DashBoard



```
Item {
    id: valueSource
    property bool useRealData: true
    property real kph: 0
    property real rpm: 1
    property real fuel: 0.85
    Connections {
        target: tcpClient
        onTelemetryUpdated: function(rpm0to6, speedKmh, fuelL, steering) {
            if (useRealData) {
                kph = speedKmh
                rpm = rpm0to6
                fuel = Math.max(0, Math.min(1, fuelL / 1400.0))
            }
        }
    }
}
```

QT cpp code – Properties of Item

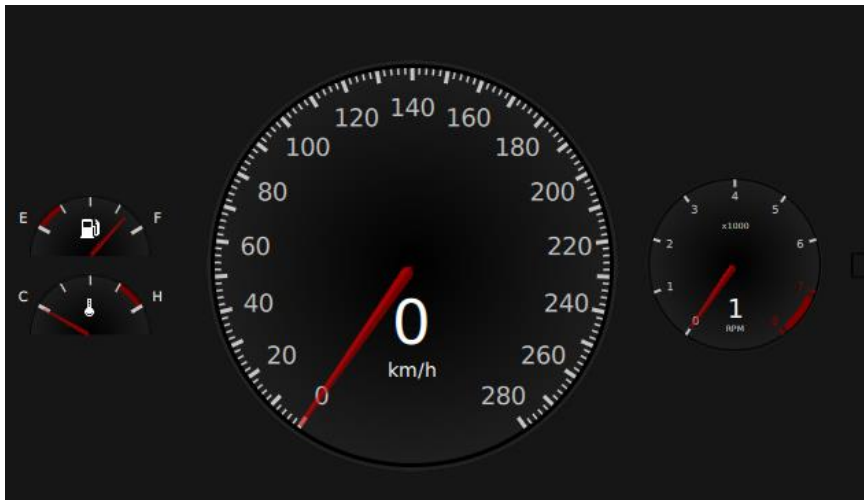
- D3-G에서 QT Dashboard와 소켓 통신
- 원하는 값만 추출하여 json 파일 형태로 QT에 전송
- QT에서는 signals를 통해 받아온 값을 qml로 전송
- qml에서 받아온 값들을 활용해 rpm, speed, fuel 값들을 실시간으로 시각화

D3-G에서 Dashboard 실행

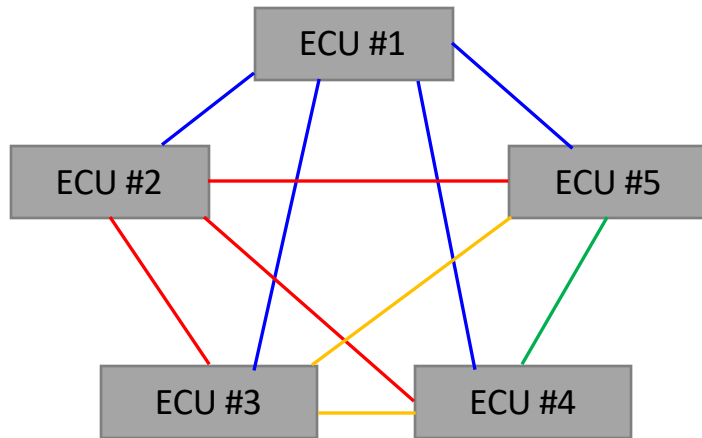
- QT package install
 - `$ apt install qml-module-qtquick-extras`

D3-G에서 Dashboard 실행

- D3-G 에서 앞에서 작성한 파이썬 코드 실행
 - `$ cd Step_2`
 - `$ python3 Data_parsing_QT.py`
- 준비한 qt 앱 실행
 - `$ chmod 755 dashboard`
 - `$ ./dashboard`



Overview of CAN

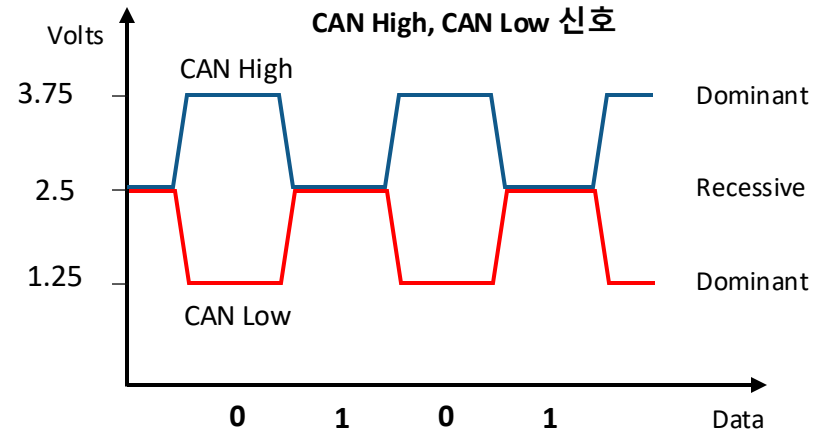
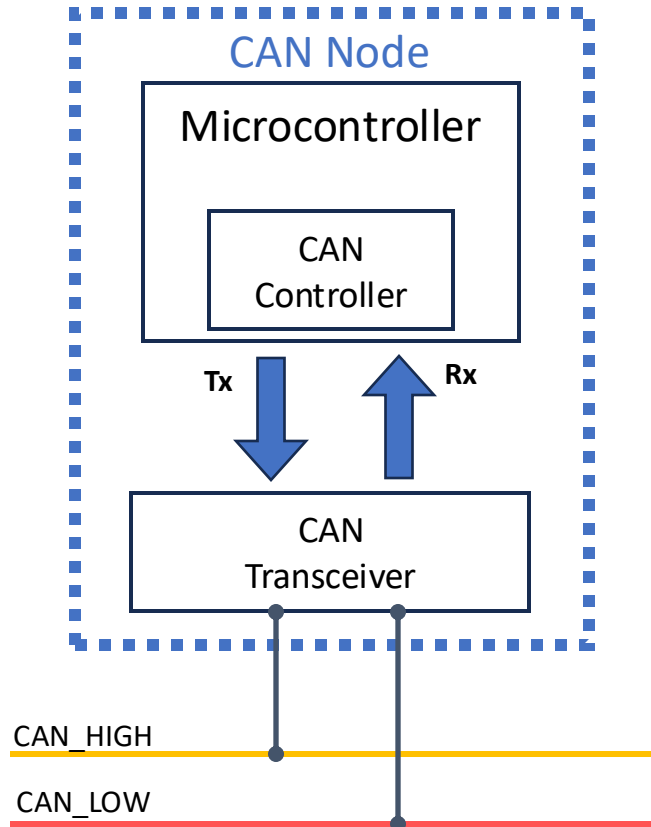
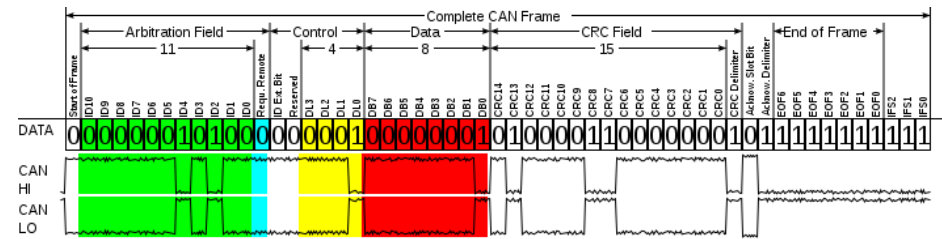


- 제어기가 추가될 때마다 통신라인 추가
- 비용과 함께 차량 무게 증가
 - 생산성 & 안전성 부적합



- CAN : **C**ontroller **A**rea **N**etwork(1983, BOSCH)
- Bus Topology 구조
- Broadcasting 방식의 통신

Overview of CAN

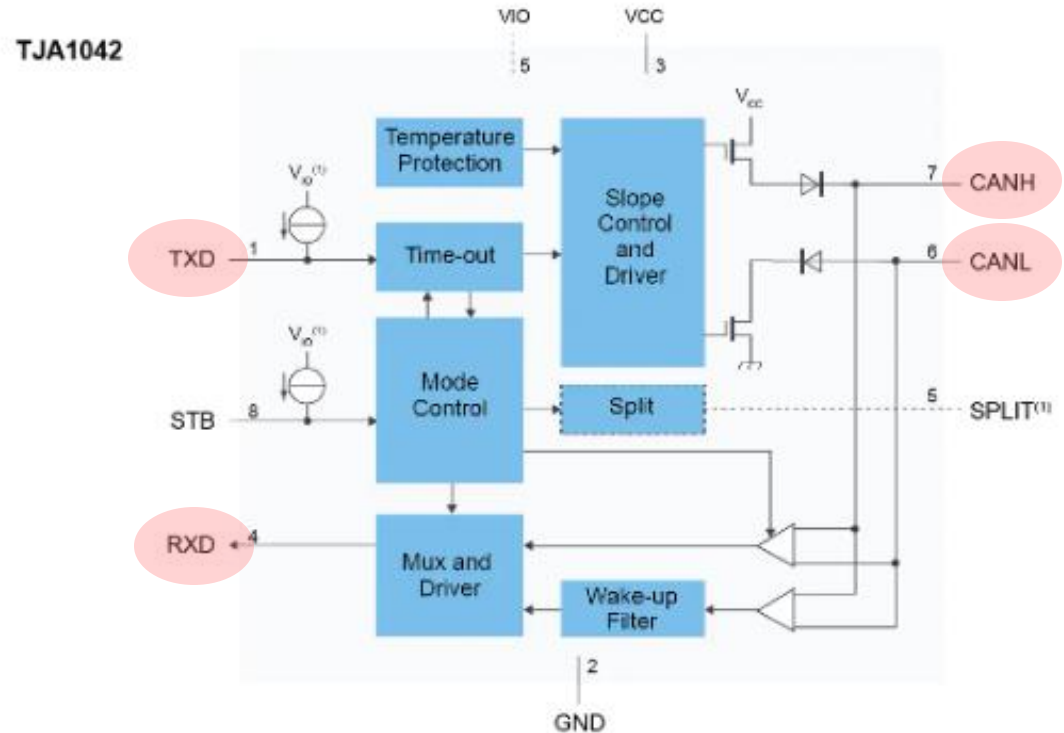


- CAN Controller는 Tx Pin을 통해 비트스트림 데이터를 트랜시버에 전달
- CAN Transceiver 로 CAN HIGH, CAN LOW 핀으로 실제 전압(차동 신호)로 변환
 - 반드시 **Transceiver 장비 필요**
- CAN High 와 CAN Low 간 전압차를 이용하여 0/1 결정
 - CAN High – CAN low : 0.9 ~ 5 V = 0 (Dominant)
 - CAN High – CAN low : -0.1 ~ 0.5 V = 1 (Receive)

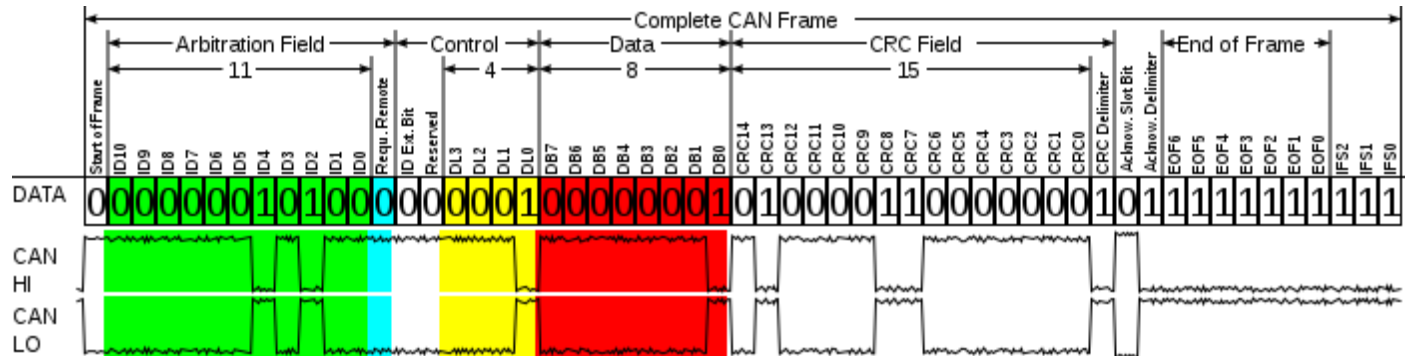
Overview of CAN



Trasceiver 에 들어가는 칩셋



Overview of CAN



설정한 ID 값은 Arbitration Field

Data Length 값은 Control Field

Data 값은 Data Field

에 각각 대응되어서 데이터가 채워지고 CAN Frame 이 완성됨

Overview of CAN

CAN DB Example

Message ID	Message Name	Data Length (byte)	Transmitter	Receiver	Period
0x123	Steering_status	4	Steering Controller	xx,xx,xx	100ms
0x200	Engine_status	3	Engine Controller	xx,xx,xx	200ms
0x300	Battery_status	4	BMS	xx,xx,xx	100ms

Message Signal Example

Message ID	Signal List
0x300	Battery Voltage
	Battery Max Current
	Battery Capacity

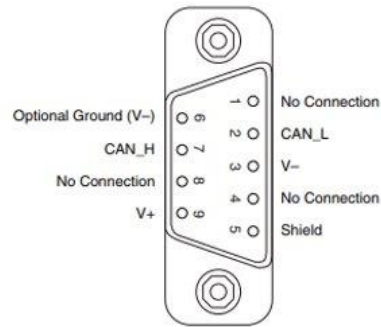
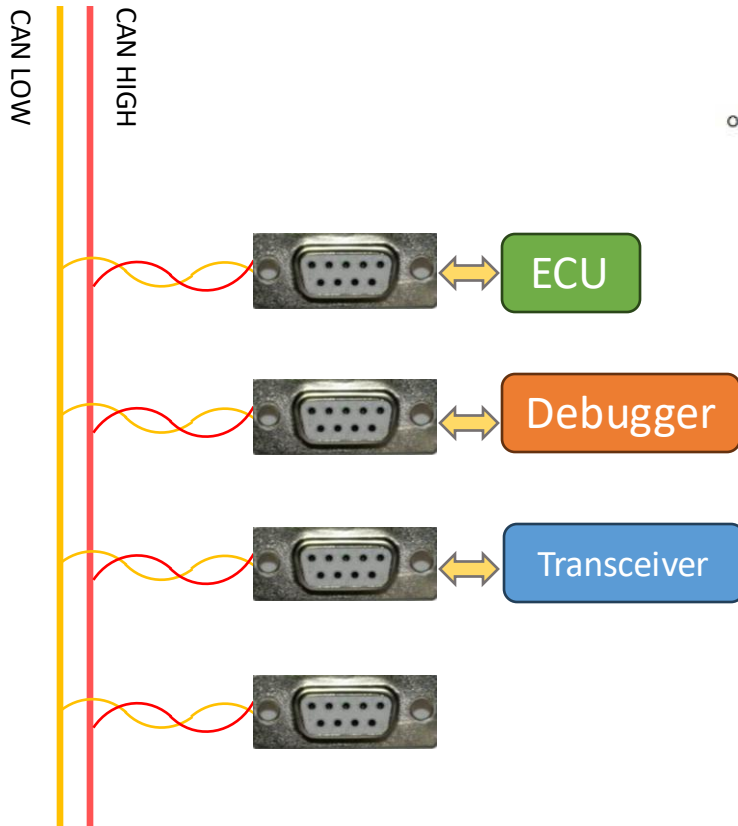
Battery Voltage Signal Detail Information

- Start bit : 0
- Length : 6
- Unit : V
- ...

Bit Position							
7	6	5	4	3	2	1	0
		Battery Voltage					
	Battery Max Current						
Battery Capacity							

- CAN DB (DBC) 차량제조사(OEM)에 의해 정의됨
- 정의된 DBC를 각 차량 부품사가 공급받아 전장시스템 연구&개발

Overview of CAN



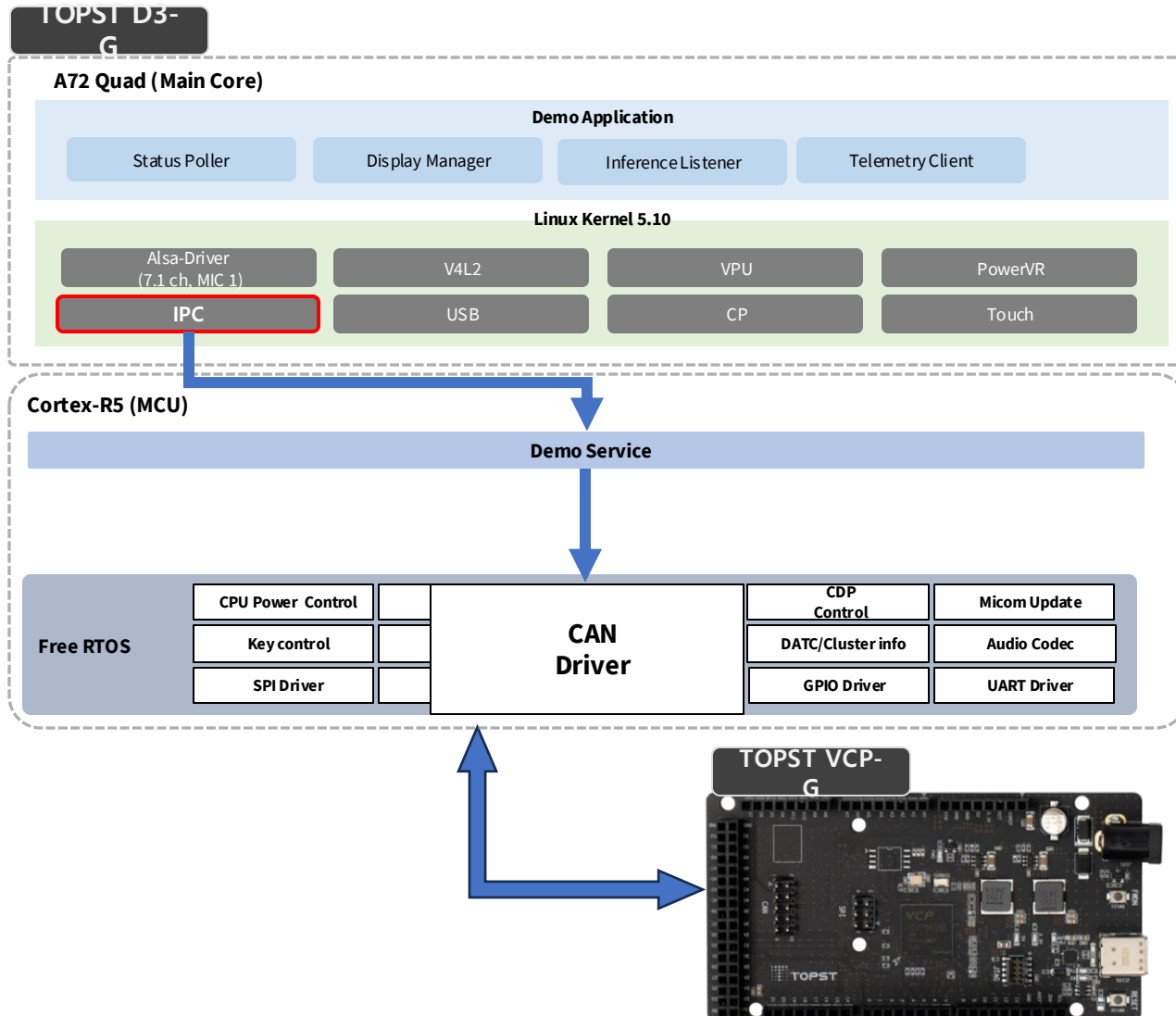
D-sub 9 pin port



Examples of CAN Transceiver

- CAN 통신을 위한 표준 규격 포트 : D-sub
- D-sub 9 pin 내 CAN_H, CAN_L port hole 로 ECU Transiver 혹은 CAN Debugger(CANoe, Vector 사) 장비 연결 가능
- 본 실습에서 0번 채널에 연결되어 있는 보드끼리 통신
 - [Transmitter] CAN 0 ch ↔ [Receiver] CAN 0 ch

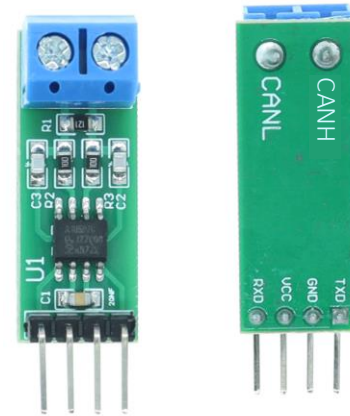
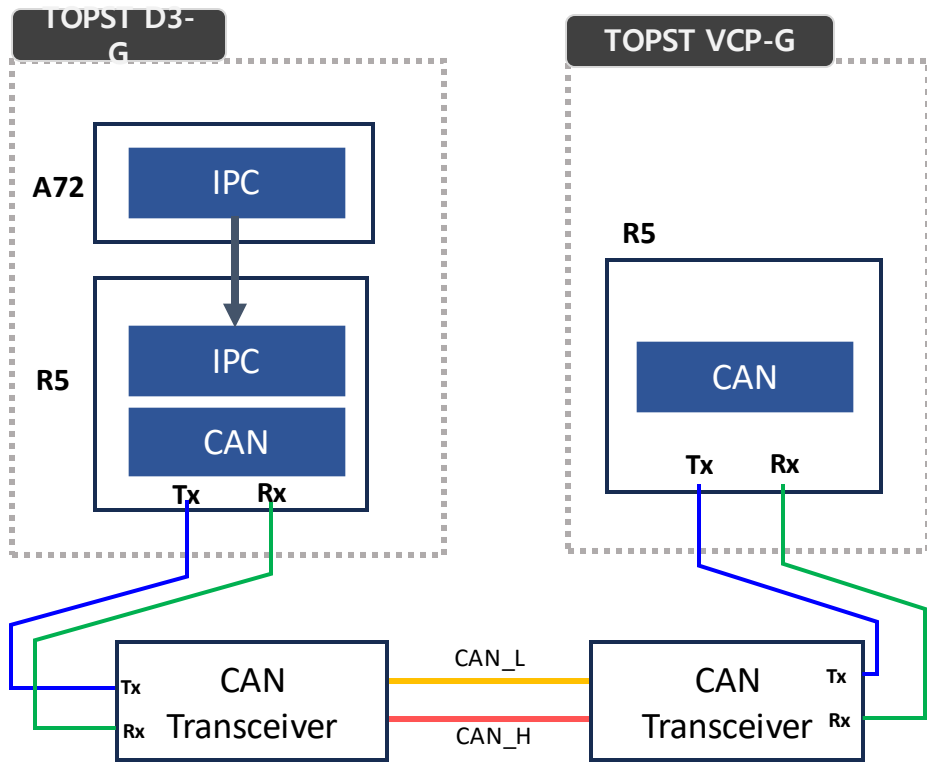
TOPST 보드간 CAN 메시지 전송 흐름



*IPC : Inter Processor Communication

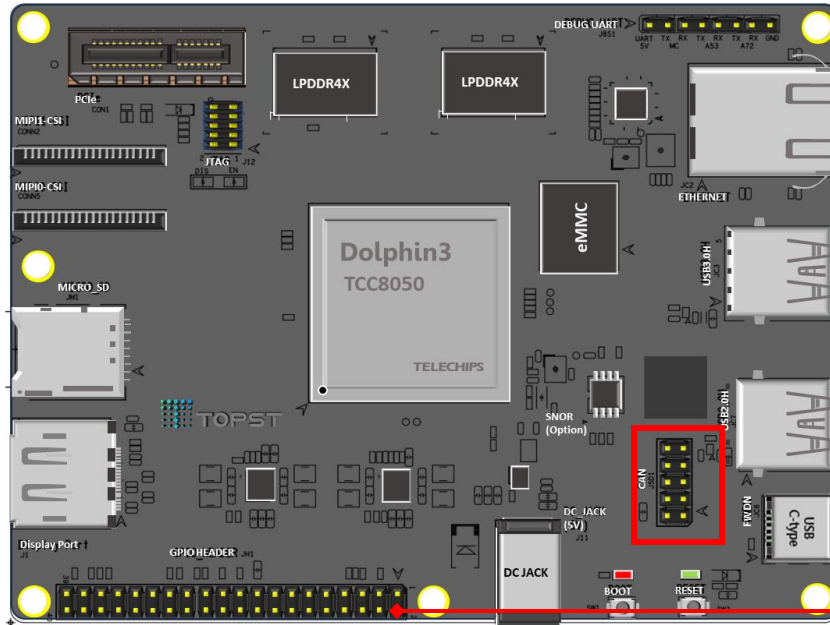
1. D3-G to Micom IPC를 이용해 상태 전달
 - CAN Message와 유사한 별도의 User-defined protocol을 사용
2. Micom에서 user-defined protocol을 수신하고 해석
3. Micom에서 CAN 통신 포트 설정 (init, open 등)
4. 전송할 항목을 CAN 통신 프로토콜로 전송

TOPST 보드간 CAN 메시지 전송 흐름

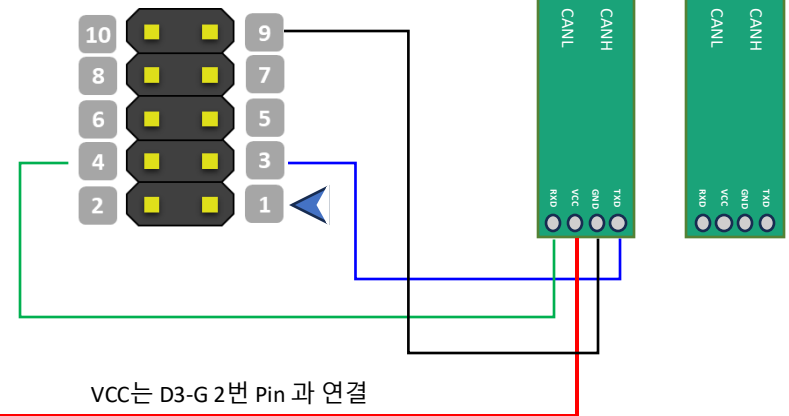


- 본 실습에서 사용될 CAN Transceiver
 - TJA1050 CAN 모듈

D3-G 보드와 CAN Transceiver 연결



CAN Header Pinout

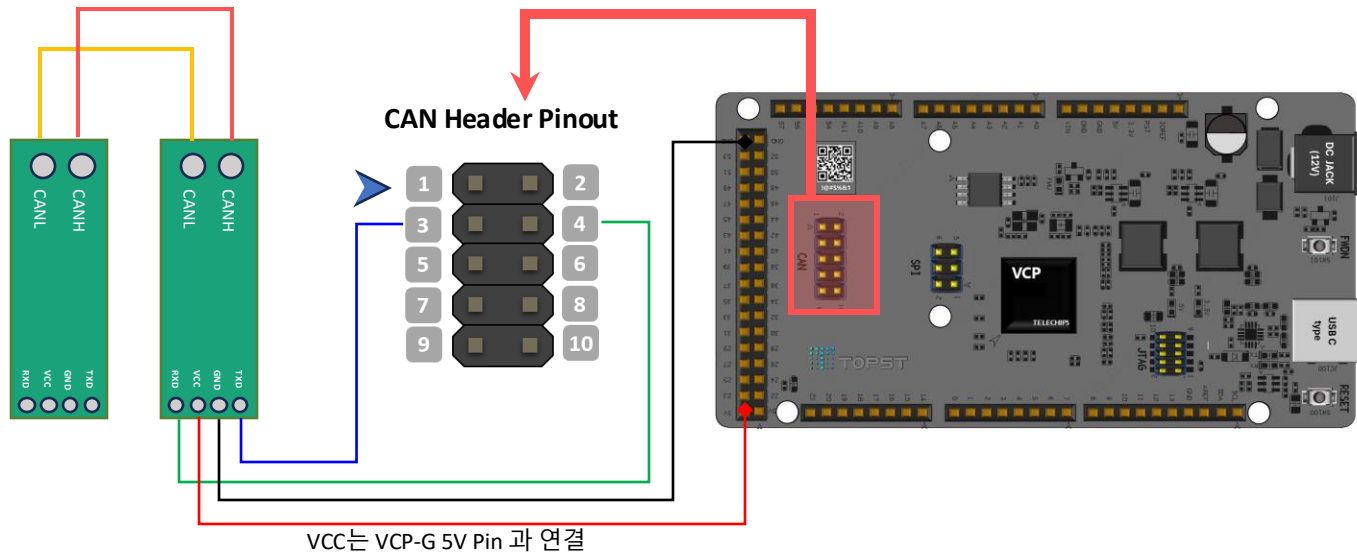


Header Pinout (2.54mm)	Pin No	NET	Function
	1,2	SYS_3P3	3.3V Power
	3	CAN0_TX	
	4	CAN0_RX	
	5	CAN1_TX	
	6	CAN1_RX	
	7	CAN2_TX	
	8	CAN2_RX	
	9,10	DGND	Ground

CAN Header Pinout

- 총 10개의 pinout 으로 구성된 CAN pin header 사용
- 본 실습에선 **CAN 0** 채널만 사용하여 실습
- 제공된 CAN Transceiver는 전원 supply를 5 v를 가해야 동작하므로 D3-G GPIO 40 pin 배열 중 2번 pin 에 연결해야 함

VCP-G 보드와 CAN Transceiver 연결



Header Pinout (2.54mm)	Pin No	NET	Function
	1,2	SYS_3P3	3.3V Power
	3	CAN0_TX	
	4	CAN0_RX	
	5	CAN1_TX	
	6	CAN1_RX	
	7	CAN2_TX	
	8	CAN2_RX	
	9,10	DGND	Ground

CAN Header Pinout

- 총 10개의 pinout 으로 구성된 CAN pin header 사용
- 본 실습에선 **CAN 0** 채널만 사용하여 실습
- 제공된 CAN Transceiver는 전원 supply를 5 V를 가해야 동작하므로 VCP-G 배열 중 5 V pin 에 연결해야 함

AP에서 MCU로 메시지 보내기(A72→R5)

```
root@d3-g-topst-main:/dev# ls -al /dev/tcc*
crw-rw---- 1 root video 248, 0 Apr 29 2022 /dev/tcc_dptx_audio
crw----- 1 root root   10, 57 Apr 29 2022 /dev/tcc_hsm
crw----- 1 root root  244, 0 Apr 29 2022 /dev/tcc_ipc_ap
crw----- 1 root root  245, 0 Apr 29 2022 /dev/tcc_ipc_micom
crw-rw---- 1 root video  10, 60 Apr 29 2022 /dev/tcc_lut
crw-rw---- 1 root video  10, 59 Apr 29 2022 /dev/tcc_lut_3d_d0
crw-rw---- 1 root video  10, 58 Apr 29 2022 /dev/tcc_lut_3d_d1
crw----- 1 root root   10, 52 Apr 29 2022 /dev/tccmisc
```

- D3-G 의 /dev/tcc_ipc_micom 이용

- \$ git clone <https://github.com/topst-development/Education.git>
 - d3-g app /IPC_Example.py 참고
- IPC_Example.py - MCU로 메시지 전송 코드 예제
- IPC_Library.py - MCU로 전송을 담당하는 library 들

AP에서 MCU로 메시지 보내기 (A72→R5)

```
topst@TOPST:~/ipc$ sudo python3 ./IPC_Example.py --help
usage: IPC_Example.py [-h] [--file_path FILE_PATH] [--channel CHANNEL] [--txOnly TXONLY] [--canID CANID] [--sndDataHex SNDDATAHEX] [--sndData SNDDATA]
                        {snd,rev,sndrecv}

IPC Sender/Receiver

positional arguments:
  {snd,rev,sndrecv}      'snd': send, 'rev': receive, 'sndrecv': send then receive once

options:
  -h, --help            show this help message and exit
  --file_path FILE_PATH File path for IPC communication
  --channel CHANNEL     can channel bit flag: b111
  --txOnly TXONLY       tx only channel bit flag: b111 (dec: 1, 2, 4)
  --canID CANID         CAN ID max 16-bit: e.g. 123 or 0x1234
  --sndDataHex SNDDATAHEX Value for sndData as a hex string, e.g., '12345678' (default: string)
  --sndData SNDDATA     Default data if no sndDataHex provided
topst@TOPST:~/ipc$ ;2~;4~
```

• IPC_example.py 실행

- 전송모드 snd, recv, sndrecv 명시 가능
- --channel : CAN 통신 채널을 의미하며 1, 2, 4는 각각 0번, 1번, 2번 채널을 의미
- --canID : CAN 노드의 Identifier 를 의미
- --sndDataHex : 전송하고자 하는 전체 data를 의미하며 HEX 값으로 2자리씩 붙여서 명시한다

(ex : 0x123456 를 전송할때 '123456' 로 명시)

AP에서 MCU로 메시지 보내기 (A72→R5)

```
def main():
    parser = argparse.ArgumentParser(description="IPC Sender/Receiver")
    parser.add_argument("mode", choices=["snd", "rev", "sndrecv"], help="'snd': send, 'rev': receive, 'sndrecv': send then receive once")
    parser.add_argument("--file_path", default="/dev/tcc_ipc_micom", help="File path for IPC communication")
    parser.add_argument("--channel", type=int, default=1, help="can channel bit flag: b111")
    parser.add_argument("--txOnly", type=int, default=0, help="tx only channel bit flag: b111 (dec: 1, 2, 4) ")
    parser.add_argument("--canID", type=parse_hex16, default=0, help="CAN ID max 16-bit: e.g. 123 or 0x1234")
    parser.add_argument("--sndDataHex", type=str, help="Value for sndData as a hex string, e.g., '12345678' (default: string)")
    parser.add_argument("--sndData", type=str, default="12345678", help="Default data if no sndDataHex provided")

    args = parser.parse_args()
    print(f"args.mode: {args.mode}, args.channel: {args.channel}, args.txOnly: {args.txOnly}, args.canID: 0x{args.canID:04X}")

    channel_str = ", ".join(parse_channels(channel_bitmask))
    txonly_str = ", ".join(parse_channels(tx_only_channel_bitmask))
    print(f"channel_bitmask: 0x{channel_bitmask:02X} (channels: {channel_str or 'None'})")
    print(f"tx_only_channel_bitmask: 0x{tx_only_channel_bitmask:02X} (channels: {txonly_str or 'None'})")
    print(f"canID: 0x{canID:04X}")
    print(f"sndData: {sndData}")
    print(f"sndDataLength: {sndDataLength}")

    ret = IPC_SendPacketwithIPCHheader(args.file_path, channel_bitmask, tx_only_channel_bitmask, canID, sndData)
```

• 제공된 IPC python 예제

- --file_path 는 /dev/tcc_ipc_micom 을 기본으로 사용
- IPC_SendPacketwithIPCHheader 함수를 통해, canID, sndData 전송에 중점

AP에서 MCU로 메시지 보내기

- **can_demo.c**
 - WSL 환경에서 R5(MCU) 소스코드에서 확인가능
 - \$ git clone <https://github.com/topst-development/D3G-R5.git>
 - \$ cd ~/D3G-R5/sources/app.sample/app.can.demo
 - python 예제 코드에서 전달한 message format을 해석
 - can_id와 canData를 추출

```
static void CAN_Edu_IpcCbFunc(uint16 uhwCmd, uint8 *pucData, uint16 uhwLength)
{
    if (uhwCmd == (uint16)IPC_IPC_CMD_CA72_EDUCATION_CAN_DEMO_START)
    {
        if (uhwLength == 0)
        {
            CAN_E("%s:%d uhwLength invalid !!!\n", __func__, uhwLength);
            return;
        }

        uint8 ch_bitmask = (pucData[0] & 0xFF);
        uint8 tx_only_bitmask = (pucData[1] & 0xFF);
        uint16 can_id = (pucData[2] << 8) | pucData[3];

        CAN_D("\nch_bitmask: 0x%X, tx_only_bitmask: 0x%X, can_id: 0x%X\n", ch_bitmask, tx_only_bitmask, can_id);

        for (uint8 ch = 0; ch < CAN_CONTROLLER_NUMBER; ch++)
        {
            if ((ch_bitmask & CAN_DEMO_CHANNEL_VALUE_LIST[ch]) == 0)
                continue;

            uint16 cmd_len = 4;
            CANDemoValue_t *canData = &gDemoData[ ch ].value;
            canData->id = can_id;

            canData->length = uhwLength - cmd_len;
            SAL_MemCopy(canData->data, &pucData[cmd_len], canData->length);
        }
    }
}
```


CAN 메시지 테스트 시나리오 선택

Telemetry Information through Rest API

```
{
  "game": {
    "connected": true,
    "gameName": "ETS2",
    "paused": false,
    "time": "0001-01-01T11:27:00Z",
    "timeScale": 3,
    "nextRestStopTime": "0001-01-01T09:40:00Z",
    "version": "1.18",
    "telemetryPluginVersion": "11",
    "serverLang": "ko-KR"
  },
  "truck": {
    "id": "daf",
    "make": "DAF",
    "model": "XF",
    "speed": 0.001113137,
    "cruiseControlSpeed": 0,
    "cruiseControlOn": false,
    "odometer": 40.3591156,
    "gear": 0,
    "displayedGear": 1,
    "forwardGears": 12,
    "reverseGears": 4,
    "shifterType": "arcade",
    "engineRpm": 552.8489,
    "engineRpmMax": 2200,
    "fuel": 1379.04224,
    "fuelCapacity": 1400,
    "fuelAverageConsumption": 0.5267514,
    "fuelWarningFactor": 0.15,
    "fuelWarningOn": false,
    "wearEngine": 0.06315411,
    "wearTransmission": 0.0376851037,
    "wearCabin": 0.128981233,
    "wearChassis": 0.161226511,
    "wearWheels": 0.0254478715,
    "userSteer": 0,
    "userThrottle": 0,
    "userBrake": 0,
    "userClutch": 0,
    "gameSteer": -0.136845276,
    "gameThrottle": 1.401298e-45,
    "gameBrake": 1,
    "gameClutch": 0,

```

```
"acceleration": {
  "x": -0.000117638265,
  "y": 0.00023695844,
  "z": 0.000205035089
},
"head": {
  "x": 0.728278756,
  "y": 1.4439919,
  "z": 0.341633439
},
"cabin": {
  "x": 0,
  "y": 1.23489118,
  "z": -2.23568821
},
"hook": {
  "x": 0,
  "y": 1,
  "z": -5.619217
},
"trailer": {
  "attached": true,
  "id": "watertank",
  "name": "Water Tank",
  "mass": 2100,
  "wear": 0.0391398557,
  "placement": {
    "x": -53894.2,
    "y": 23.820528,
    "z": -14653.6982,
    "heading": 0.244444042,
    "pitch": -0.00113043422,
    "roll": 0.00007687757
  }
},
"job": {
  "income": 902,
  "deadlineTime": "0001-01-01T15:43:00Z",
  "remainingTime": "0001-01-01T04:16:00Z",
  "sourceCity": "Cardiff",
  "sourceCompany": "Steinbruch",
  "destinationCity": "Cardiff",
  "destinationCompany": "Stokes"
},
"navigation": {
  "estimatedTime": "0001-01-01T00:01:29Z",
  "estimatedDistance": 714,
  "speedLimit": 48
}
}
```

• 시나리오 선택 (예)

- 핸들 및 페달 상태 표시
- 차량 상태 값
 - 전조등
 - 방향 지시등
 - 브레이크등
 - 비상지시등
 - 연료레벨

- 위의 값 외에 구현하고자 하는 시나리오에 맞춰 API 값 선택 가능

VCP로 전송할 CAN 메시지 구성

- 실습용 CAN message 정의

메시지 종류	ID	Data
브레이크등	0x101	[0] : on-0x01, off-0x02
방향지시등	0x102	[0] : on-0x01, off-0x02 [1] : left-0x01, right-0x02
비상지시등	0x103	[0] : on-0x01, off-0x02
전조등	0x104	[0] : on-0x01, off-0x02
연료레벨	0x105	[0] : 0%-0x00 ~ 100%-0x64
모터동작	0x106	[0] : 0kph-0x00 ~ 100kph-0x64

D3-G MCU 내 CAN 메시지 처리

- CAN message 만들기
 - MCU가 AP로 부터 수신한 can_id 와 canData를 사용해 CAN 통신 프로토콜을 생성

MCU에서 사용하는 CAN Message Format

```
typedef struct CANMessage
{
    CANMessageBufferType_t    mBufferType;
    uint8                     mBufferIndex;
    uint8                     mErrorStateIndicator;
    uint8                     mExtendedId;
    uint8                     mRemoteTransmitRequest;
    uint32                    mId;
    uint8                     mFDFormat;
    uint8                     mBitRateSwitching;
    uint8                     mMessageMarker;
    uint8                     mEventFIFOControl;
    uint8                     mDataLength; //byte unit
    uint8                     mData[CAN_DATA_LENGTH_SIZE];
} CANMessage_t;
```

CAN Message Format에 고정값 할당의 예

```
#if 1 /* Misra2012:9.3 - Partially Uninitialized Array */
/* BufferType, Index, ESI, ExtendedID, RTR, ID, FD, BRS, MM, EventFIFO, DLC, DATA */
{ CAN_TX_BUFFER_TYPE_DBUFFER, 0, 0, 0, 0, 0x11, 0, 0, 0xFF, 1, 1, {0, /* Data is defined as much as DLC in sending function */ } },
{ CAN_TX_BUFFER_TYPE_DBUFFER, 1, 0, 1, 0, 0x22, 0, 0, 0xFF, 1, 2, {0, /* Data is defined as much as DLC in sending function */ } },
{ CAN_TX_BUFFER_TYPE_DBUFFER, 2, 0, 0, 0, 0x33, 0, 0, 0xFF, 1, 3, {0, /* Data is defined as much as DLC in sending function */ } },
{ CAN_TX_BUFFER_TYPE_DBUFFER, 3, 0, 1, 0, 0x44, 0, 0, 0xFF, 1, 4, {0, /* Data is defined as much as DLC in sending function */ } },
{ CAN_TX_BUFFER_TYPE_FIFO, 0, 0, 0, 0, 0x55, 0, 0, 0xFF, 1, 5, {0, /* Data is defined as much as DLC in sending function */ } },
{ CAN_TX_BUFFER_TYPE_FIFO, 0, 0, 1, 0, 0x66, 0, 0, 0xFF, 1, 6, {0, /* Data is defined as much as DLC in sending function */ } },
{ CAN_TX_BUFFER_TYPE_FIFO, 0, 0, 0, 0, 0x77, 0, 0, 0xFF, 1, 7, {0, /* Data is defined as much as DLC in sending function */ } },
{ CAN_TX_BUFFER_TYPE_FIFO, 0, 0, 1, 0, 0x88, 0, 0, 0xFF, 1, 8, {0, /* Data is defined as much as DLC in sending function */ } },
{ CAN_TX_BUFFER_TYPE_FIFO, 0, 0, 0, 0, 0x99, 0, 0, 0xFF, 1, 8, {0, /* Data is defined as much as DLC in sending function */ } },
```

D3-G MCU 내 CAN 메시지 처리

```

CANErrorType_t CAN_Init
(
    void
)
{
    uint8          ucCh;
    CANController_t * psControllerInfo;
    CANErrorType_t  ret;

    psControllerInfo = NULL_PTR;
    ret = CAN_ERROR_NONE;

    for( ucCh = 0 ; ucCh < CAN_CONTROLLER_NUMBER ; ucCh++ )
    {
        psControllerInfo = &CANDrvierInfo.dControllerInfo[ ucCh ];
        psControllerInfo->cChannelHandle = ucCh;

        ( void ) CAN_PortingDisableControllerInterrupts( psControllerInfo->cChannelHandle );

        ret = CAN_DrvInitChannel( psControllerInfo );

        if( ret != CAN_ERROR_NONE )
        {
            CAN_D( "[CAN]CAN_Init Failed \r\n", psControllerInfo->cChannelHandle );
            break;
        }

        ( void ) CAN_PortingEnableControllerInterrupts( psControllerInfo->cChannelHandle );
    }

    if( ret == CAN_ERROR_NONE )
    {
        CAN_D( "[CAN]CAN_Init Completed \r\n", psControllerInfo->cChannelHandle );
    }

    return ret;
} « end CAN_Init »

```

• CAN port 초기화

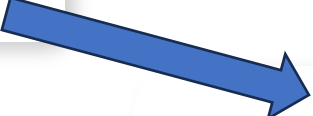
- Driver Level 초기화
- Register 직접 Setup
- 3채널 GPIO 초기화
- Controller Clock 초기화

D3-G MCU 내 CAN 메시지 처리

- CAN 메시지 전송

- 생성한 메시지를 Driver API에 전달

```
/* Send Tx message */  
psTxMsg = &sTxMessageInfo[ uiTxMsgCnt ];  
  
( void ) CAN_SendMessage( ucCh, psTxMsg, &ucTxBufferIndex );
```



```
if( ucCh < CAN_CONTROLLER_NUMBER )  
{  
    psControllerInfo = &CANDrvierInfo.dControllerInfo[ ucCh ];  
  
    if( psMsg != NULL_PTR )  
    {  
        if( ( psControllerInfo->cMode == CAN_MODE_OPERATION )  
            || ( psControllerInfo->cMode == CAN_MODE_MONITORING )  
            || ( psControllerInfo->cMode == CAN_MODE_INTERNAL_TEST )  
            || ( psControllerInfo->cMode == CAN_MODE_EXTERNAL_TEST ) )  
        {  
            ret = CAN_MsgSetTxMessage( psControllerInfo, psMsg, pucTxBufferIndex );  
        }  
        else  
        {  
            ret = CAN_ERROR_CONTROLLER_MODE;  
        }  
    }  
    else  
    {  
        ret = CAN_ERROR_NOT_INIT;  
    }  
}  
} « end if ucCh<CAN_CONTROLLER_N... »
```

D3-G MCU 내 CAN 메시지 처리

• CAN 메시지 전송

- 보내고자 하는 CAN Message를 Register에 직접 Deploy

```

if( psMsg->mBufferType == CAN_TX_BUFFER_TYPE_FIFO )
{
    ucReqIndex = ( uint8 ) psControllerReg->crTxFIFOQueueStatus.rFReg.rFTFQPI;
    uiTxMsgAddr = psControllerInfo->cRamAddressInfo.raTxBufferAddr + ( sizeof( CANRamTxBuffer_t ) * ( uint32 ) ucReqIndex );
    psTxMsg = ( CANRamTxBuffer_t * ) uiTxMsgAddr;

    *pucTxBufferIndex = ucReqIndex;
}
else if( psMsg->mBufferType == CAN_TX_BUFFER_TYPE_QUEUE )
{
    ucReqIndex = psMsg->mBufferIndex + ( uint8 ) psControllerReg->crTxFIFOQueueStatus.rFReg.rFTFQPI;
    uiTxMsgAddr = psControllerInfo->cRamAddressInfo.raTxBufferAddr + ( sizeof( CANRamTxBuffer_t ) * ( uint32 ) ucReqIndex );
    psTxMsg = ( CANRamTxBuffer_t * ) uiTxMsgAddr;

    *pucTxBufferIndex = ucReqIndex;
}
else if( psMsg->mBufferType == CAN_TX_BUFFER_TYPE_DBUFFER )
{
    ucReqIndex = psMsg->mBufferIndex;
    uiTxMsgAddr = psControllerInfo->cRamAddressInfo.raTxBufferAddr + ( sizeof( CANRamTxBuffer_t ) * ( uint32 ) ucReqIndex );
    psTxMsg = ( CANRamTxBuffer_t * ) uiTxMsgAddr;

    *pucTxBufferIndex = ucReqIndex;
}
else
{
    /* unknown buffer type */
    ret = CAN_ERROR_NO_MESSAGE;
}

if( ( psMsg->mBufferType == CAN_TX_BUFFER_TYPE_FIFO ) || ( psMsg->mBufferType == CAN_TX_BUFFER_TYPE_QUEUE ) || ( psMsg->mBufferType == CAN_TX_BUFFER_TYPE_DBUFFER ) )
{
    ( void ) CAN_MsgCopyTxMessage( psTxMsg, psMsg );

    if( psControllerInfo->clxBufferInfo->tblxFIFOQueueMode != CAN_TX_BUFFER_MODE_QUEUE ) {
        ret = CAN_MsgRequestTxMessage( psControllerInfo, ucReqIndex );
    }
    else {
        ret = CAN_ERROR_NONE;
    }
}

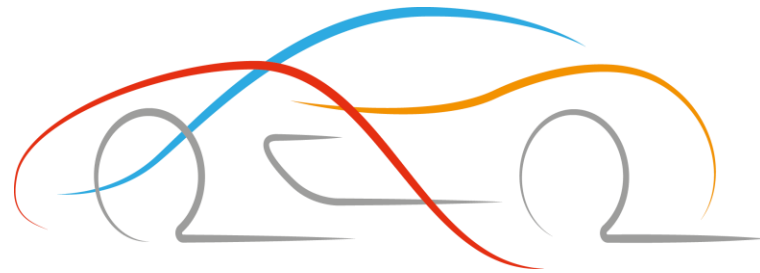
```

D3-G MCU 내 CAN 메시지 처리

• CAN 메시지 전송 후처리

- 시리얼 통신으로 연결된 Putty 로그를 통해 A72에서 전송한 CAN 메시지 확인을 아래 커맨드로 확인할 수 있다.
- `$ python3 ./IPC_Example.py snd -sndDataHex 123456`

```
topst@TOPST:~/ipc$ sudo python3 ./IPC_Example.py snd --sndDataHex 1234567
[sudo] password for topst:
args.mode: snd, args.channel: 1, args.txOnly: 0, args.canID: 0x0000
channel_bitmask: 0x01 (channels: 0)
tx_only_channel_bitmask: 0x00 (channels: None)
canID: 0x0000
sndData: b'\x124V\x07'
sndDataLength: 4
Send to ipc file_path: /dev/tcc_ipc_micom
pucData_len: 4
pucData: 12 34 56 07
Calculated CRC: B600
Packet size: 17
total size: 19
Packet data before sending:
FF 55 AA 00 05 00 01 00 08 01 00 00 00 12 34 56 07 B6 00
```



Thank You